



TAREA 2
CAJERO AUTOMÁTICO

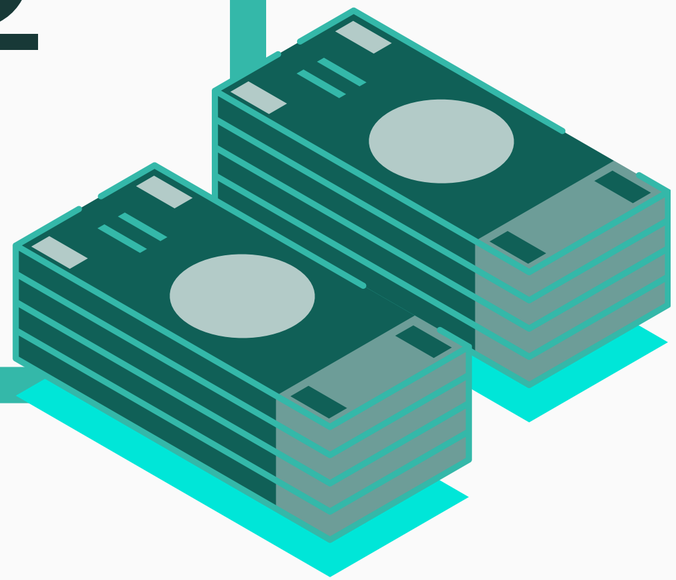
Estudiante:

Dylan Garos Quesada

Docente

Verónica Mora Lezcano





Enunciado del problema

Un cajero automático permite al usuario realizar varios retiros consecutivos sobre un mismo saldo inicial hasta que decida salir o hasta que no tenga fondos suficientes.

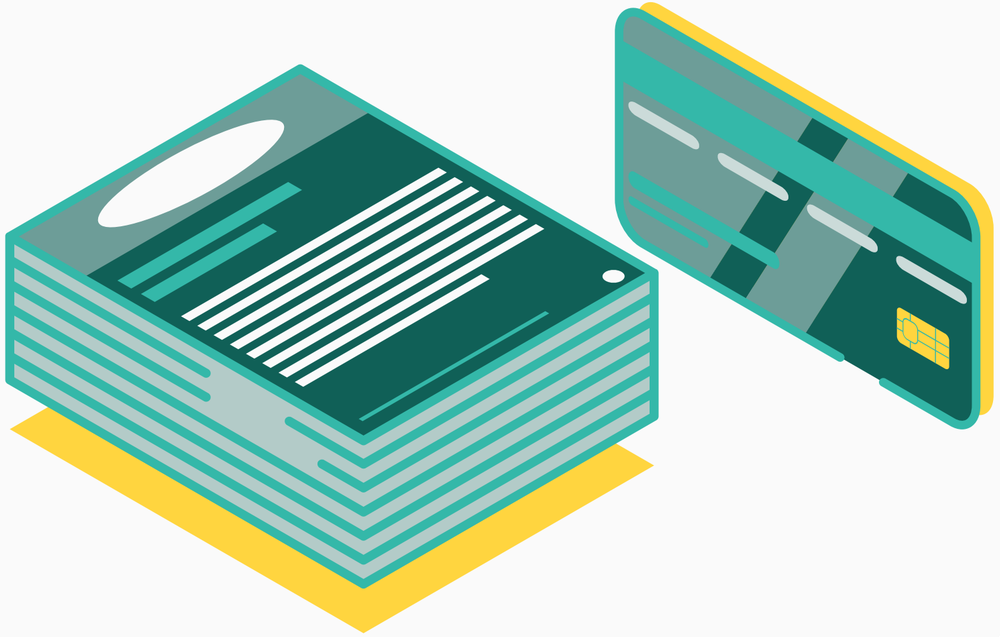
El saldo inicial de la cuenta es de ₡250 000.

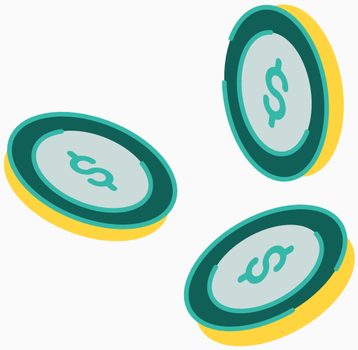
El programa debe solicitar el monto que se desea retirar y la confirmación para continuar realizando retiros. Cada operación debe cumplir las siguientes reglas:

- El retiro mínimo por operación es de ₡5 000.
- El retiro máximo por operación es de ₡100 000.
- Si el monto supera el saldo disponible, el retiro no se realiza y se muestra un mensaje.
- El ciclo continúa mientras el usuario escriba "s" y exista saldo suficiente para continuar.

Por cada intento, el programa debe indicar si el retiro fue aprobado o rechazado y mostrar el saldo actualizado. Al finalizar, debe presentar el saldo restante y la cantidad total de retiros exitosos.







Análisis del problema

¿Cuál es el problema?

Se necesita crear un programa que simule un cajero automático y permita realizar retiros de dinero de una cuenta, verificando que cada operación cumpla con las reglas establecidas.



¿Cómo se resolverá?

El programa utilizará variables para controlar el saldo disponible, el monto solicitado, la decisión de continuar y la cantidad de retiros exitosos.

Mediante condiciones, se comprobará que el monto sea igual o mayor a ₡5 000, no supere los ₡100 000 y no exceda el saldo disponible.

Se utilizará un ciclo while para permitir varios retiros consecutivos mientras el usuario desee continuar y haya saldo suficiente.

Cuando un retiro sea válido, se actualizará el saldo y aumentará el contador de retiros exitosos. Si el monto no cumple las condiciones, se rechazará la operación sin modificar el saldo.

Al terminar, se mostrarán el saldo final y la cantidad de retiros exitosos.

Objetivo

Desarrollar un programa en Python que permita gestionar retiros de dinero mediante constantes, variables, condiciones y un ciclo, mostrando resultados claros durante la ejecución y al finalizar.

Datos de entrada, constantes y variables

Datos de entrada

- Confirmación para continuar: el usuario ingresa s para realizar retiros o n para finalizar.
- Monto del retiro: cantidad de dinero que el usuario desea retirar, ingresada como un número entero.

Constantes

Constante	Valor	Descripción
SALDO_INICIAL	¢250 000	Dinero disponible al iniciar el programa.
RETIRO_MINIMO	¢5 000	Cantidad mínima permitida por retiro.
RETIRO_MAXIMO	¢100 000	Cantidad máxima permitida por retiro.

Variables

Variable	Valor inicial o tipo	Función
saldo	¢250 000	Guarda el dinero disponible en la cuenta.
contador	0	Cuenta los retiros exitosos.
continuar	Texto ingresado por el usuario	Indica si desea realizar retiros.
monto	Número entero	Almacena el monto solicitado en cada operación.

Tabla de datos de entrada y salida

Datos de entrada

Dato	Tipo de dato	Descripción
continuar	Cadena de texto	Indica si el usuario desea realizar un retiro (s) o finalizar (n).
monto	Entero	Cantidad de dinero que el usuario desea retirar.

Datos de salida

Dato	Tipo de dato	Descripción
Mensaje de resultado	Texto	Informa si el retiro fue aprobado o rechazado.
saldo	Entero	Muestra el saldo disponible después de cada intento.
Saldo final	Entero	Indica cuánto dinero queda al terminar el programa.
contador	Entero	Muestra la cantidad de retiros exitosos.

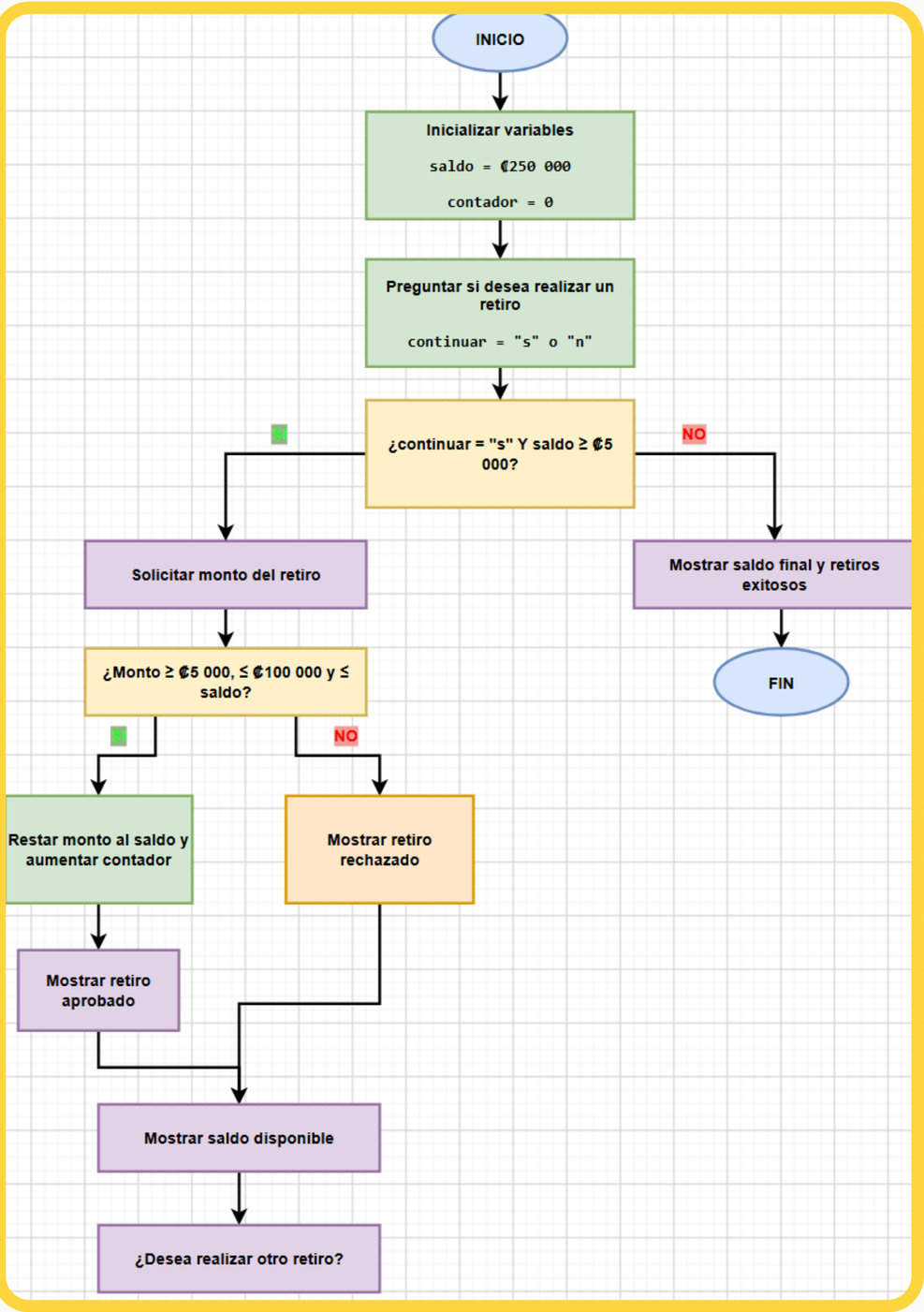


Diagrama de flujo

El siguiente diagrama representa la lógica del programa del cajero automático, desde la inicialización del saldo y el contador hasta la validación de los retiros y la presentación de los resultados finales.

Mediante decisiones y un ciclo while, el programa permite realizar varios retiros, rechaza los montos que no cumplen las condiciones establecidas y finaliza cuando el usuario decide salir o el saldo disponible es insuficiente.

Código fuente en Python

El programa se desarrolló en Python utilizando constantes, variables, estructuras condicionales y un ciclo while.
El código permite solicitar retiros, verificar que los montos cumplan las condiciones establecidas, actualizar el saldo cuando una operación es válida y contabilizar los retiros exitosos.

```python
# Constantes
SALDO_INICIAL = 250000
RETIRO_MINIMO = 5000
RETIRO_MAXIMO = 100000

# Variables iniciales
saldo = SALDO_INICIAL
contador = 0
continuar = input("¿Desea realizar un retiro? (s/n): ").lower()

# Ciclo principal
while continuar == "s" and saldo >= RETIRO_MINIMO:
    monto = int(input("¿Cuánto desea retirar? "))

    if monto >= RETIRO_MINIMO and monto <= RETIRO_MAXIMO and monto <= saldo:
        saldo = saldo - monto
        contador = contador + 1
        print("Retiro aprobado.")
    else:
        print("Retiro rechazado. Revise el monto o el saldo.")

    print("Saldo disponible:", saldo)

    if saldo >= RETIRO_MINIMO:
        continuar = input("¿Desea realizar otro retiro? (s/n): ").lower()
    else:
        continuar = "n"

# Resultado final
print("Saldo final:", saldo)
print("Retiros exitosos:", contador)
```

Tabla de seguimiento del programa

La tabla de seguimiento permite observar los cambios que ocurren en las variables durante la ejecución del cajero automático. Para este ejemplo, el usuario realiza dos retiros exitosos y después decide finalizar.

Datos de la prueba:

- Saldo inicial: ₡250 000.
- Primer retiro: ₡20 000.
- Segundo retiro: ₡10 000.
- Al final del segundo retiro, el usuario ingresa n.

Paso	Acción	Monto	Saldo	Retiros exitosos	Continuar
Inicial	Se inicializan las variables		¢250 000	0	s
1	Se aprueba el primer retiro	¢20 000	¢230 000	1	s
2	Se aprueba el segundo retiro	¢10 000	¢220 000	2	n
Final	El ciclo termina y se muestran los resultados		¢220 000	2	n

Resultado esperado

- Saldo final: ₡220 000.
- Retiros exitosos: 2.

La prueba demuestra que el programa descuenta correctamente los montos aprobados, actualiza el contador y finaliza cuando el usuario decide no continuar.

Prueba 1: caso normal con varios retiros

Objetivo

Comprobar que el programa permite realizar varios retiros consecutivos, actualiza el saldo después de cada operación y contabiliza correctamente los retiros exitosos.

Datos de prueba

- Saldo inicial: ₡250 000.
- Primer retiro: ₡20 000.
- Segundo retiro: ₡10 000.
- Decisión final: n.

Resultado obtenido

El primer retiro fue aprobado y el saldo quedó en ₡230 000. El segundo retiro también fue aprobado y el saldo quedó en ₡220 000. Finalmente, el usuario decidió no realizar más retiros.

Resultado final esperado:

- Saldo final: ₡220 000.
- Retiros exitosos: 2.

```
¿Desea realizar un retiro? (s/n): s
¿Cuánto desea retirar? 20000
Retiro aprobado.
Saldo disponible: 230000
¿Desea realizar otro retiro? (s/n): s
¿Cuánto desea retirar? 10000
Retiro aprobado.
Saldo disponible: 220000
¿Desea realizar otro retiro? (s/n): n
Saldo final: 220000
Retiros exitosos: 2
```

Conclusión

La prueba fue exitosa porque el programa aprobó los dos retiros, actualizó correctamente el saldo y mostró la cantidad total de operaciones exitosas.

Prueba 2: cero iteraciones

Objetivo

Verificar que el programa finaliza correctamente cuando el usuario decide no realizar ningún retiro desde el inicio.

Datos de prueba

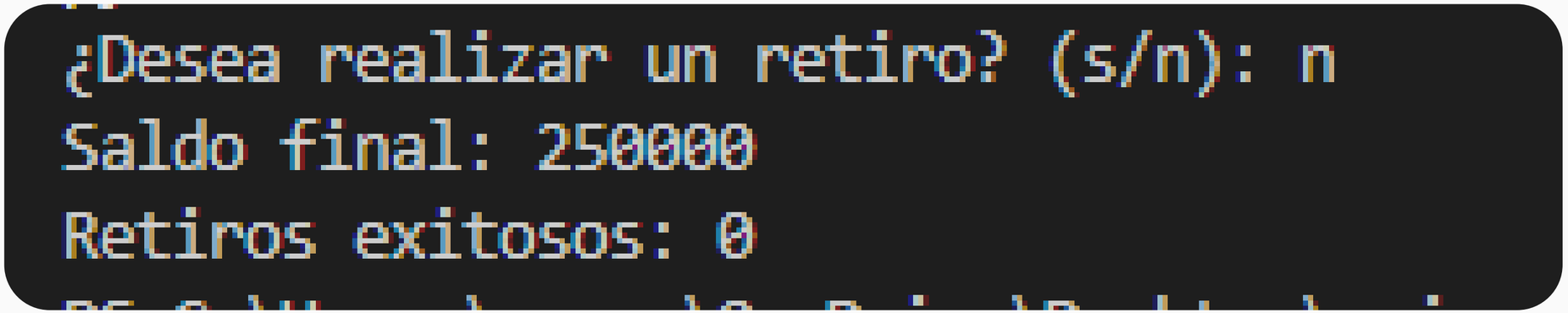
- Saldo inicial: ₡250 000.
- Decisión inicial: n.
- Cantidad de retiros realizados: 0.

Resultado obtenido

El usuario ingresó n al iniciar el programa. Por esta razón, el ciclo while no se ejecutó y el programa mostró directamente los resultados finales.

Resultado final esperado:

- Saldo final: ₡250 000.
- Retiros exitosos: 0.



Conclusión

La prueba fue exitosa porque el programa respetó la decisión del usuario, mantuvo intacto el saldo inicial y finalizó sin realizar reti

Prueba 3: una iteración

Objetivo

Comprobar que el programa permite realizar un único retiro exitoso y finaliza correctamente cuando el usuario decide no continuar.

Datos de prueba

- Saldo inicial: ₡250 000.
- Decisión inicial: s.
- Monto del retiro: ₡10 000.
- Decisión final: n.

Resultado obtenido

El usuario confirmó que deseaba realizar un retiro e ingresó ₡10 000. El programa aprobó la operación, descontó el monto del saldo inicial y solicitó la confirmación para continuar. Al ingresar n, el programa finalizó.

Resultado final esperado:

- Saldo final: ₡240 000.
- Retiros exitosos: 1.

```
¿Desea realizar un retiro? (s/n): s
¿Cuánto desea retirar? 10000
Retiro aprobado.
Saldo disponible: 240000
¿Desea realizar otro retiro? (s/n): n
Saldo final: 240000
Retiros exitosos: 1
```

Conclusión

La prueba fue exitosa porque el programa realizó un retiro válido, actualizó el saldo correctamente, incrementó el contador una vez y terminó cuando el usuario decidió salir.

Prueba 4: caso límite

Objetivo

Verificar que el programa acepta correctamente un retiro que alcanza el límite máximo permitido por operación.

Datos de prueba

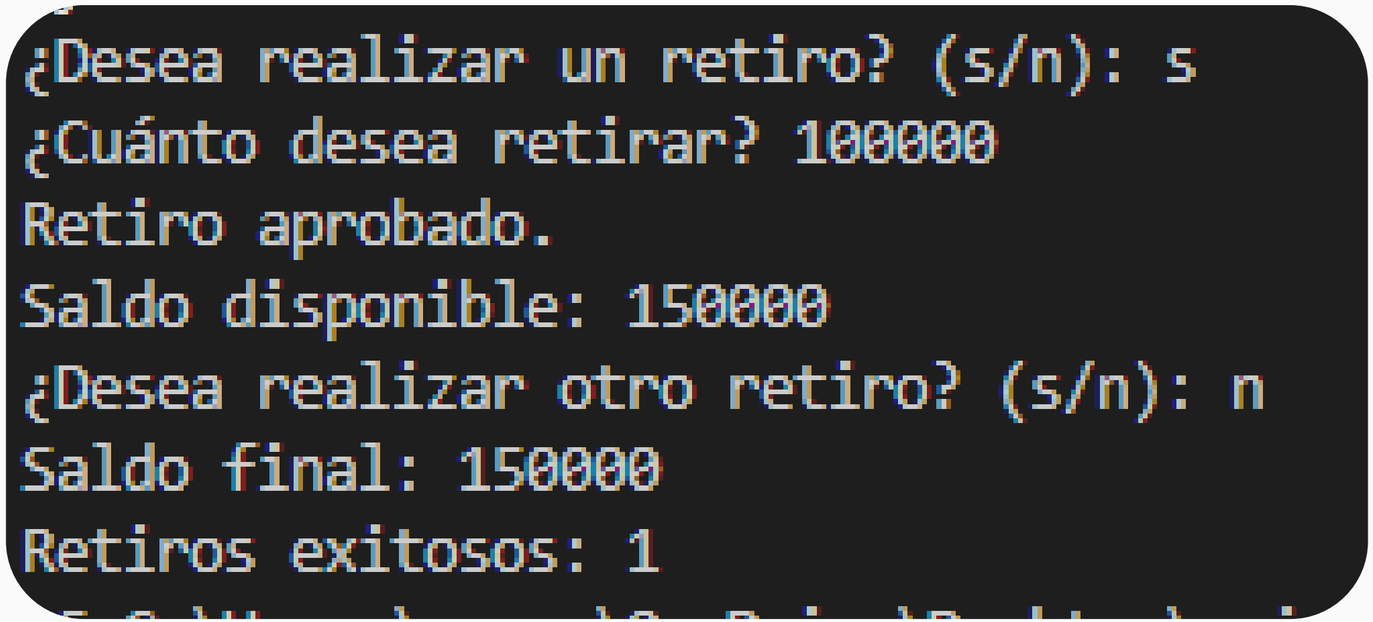
- Saldo inicial: ₡250 000.
- Decisión inicial: s.
- Monto del retiro: ₡100 000.
- Decisión final: n.

Resultado obtenido

El usuario solicitó retirar ₡100 000, monto que corresponde al límite máximo permitido. El programa aprobó la operación, descontó el dinero de la cuenta y mostró el saldo actualizado.

Resultado final esperado:

- Saldo final: ₡150 000.
- Retiros exitosos: 1.



Conclusión

La prueba fue exitosa porque el programa aceptó el monto máximo permitido, actualizó correctamente el saldo y registró un retiro exitoso.

Repositorio del proyecto

Enlace al repositorio

Archivos incluidos

- Archivo Python (.py) con el código fuente del cajero automático.
- Documento PDF con el enunciado del problema, la tabla de seguimiento y las capturas de las pruebas realizadas.

Conclusión general

El desarrollo de este programa permitió aplicar los conceptos de constantes, variables, estructuras condicionales y ciclos en Python. Además, se comprobó su funcionamiento mediante diferentes pruebas que permitieron verificar la actualización del saldo, la validación de los montos y el conteo de los retiros exitosos.

Conclusiones

- Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.
- Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.



Muchas Gracias

www.unsitiogenial.es



Recursos Gráficos

